

부품제작 · AI 활용 기술 문서

이윤수

2026년 8월 9일

CONTENTS

AI 활용 한눈에 보기	
핵심 결론	
1. AI를 어떻게 디렉팅했는가	
프롬프트보다 중요한 것은 판단 기준이었습니다	
AI 활용 전체 흐름	
제작 파이프라인	
2. 실제 프롬프트와 판단 기록	
기록 방식	
P-01. 제품 조립을 실무 문제로 만들기	
P-02. 자동 설치를 직접 조작으로 바꾸기	
P-03. 구매를 3D 결과와 연결하기	
P-04. “실제 같지 않다”를 3D 파이프라인으로 해결하기	
P-05. 생활 맥락이 틀리면 완성 기능도 버리기	
P-06. 같은 드래그 게임을 복제하지 않기	
P-07. 읽기 쉽게 만들되 3D를 가리지 않기	
P-08. 마지막 날에도 실행 경로를 다시 검증하기	
3. 기술 파이프라인	
런타임 구조	
제품 정보가 게임 규칙이 되는 과정	
Neural4D x Blender 하이브리드 자산	
설치 전후 상태를 분리한 케이블	
로딩과 성능 최적화	
4. 실패, 검증, 책임	
AI의 첫 답을 그대로 쓰지 않은 기록	
자동 검증	
사람과 AI의 역할 구분	
보안과 재현성	
외부 도구·자산·권리 표시	
현재 한계와 다음 단계	
제가 NAN 2026에서 증명하고 싶은 것	
참고 링크	

AI 활용 한눈에 보기

부품제작 에서 AI는 버튼 하나로 게임을 만들어 준 도구가 아닙니다. 저는 AI에게 문제를 조사시키고, 결과를 실제 화면에서 비판하고, 다시 만들게 하고, 테스트로 증명하게 했습니다. AI가 만든 첫 답을 채택한 것이 아니라 더 좋은 다음 질문을 설계한 과정이 이 작품의 핵심 AI 활용입니다.

NAN 2026 Game x AI Hackathon 사전 과제

문서 유형: AI 활용 기술 문서
개인 참가자: 이윤수
작성일: 2026년 8월 9일
플레이: [웹에서 바로 실행](#)
저장소: [GitHub 소스 코드](#)

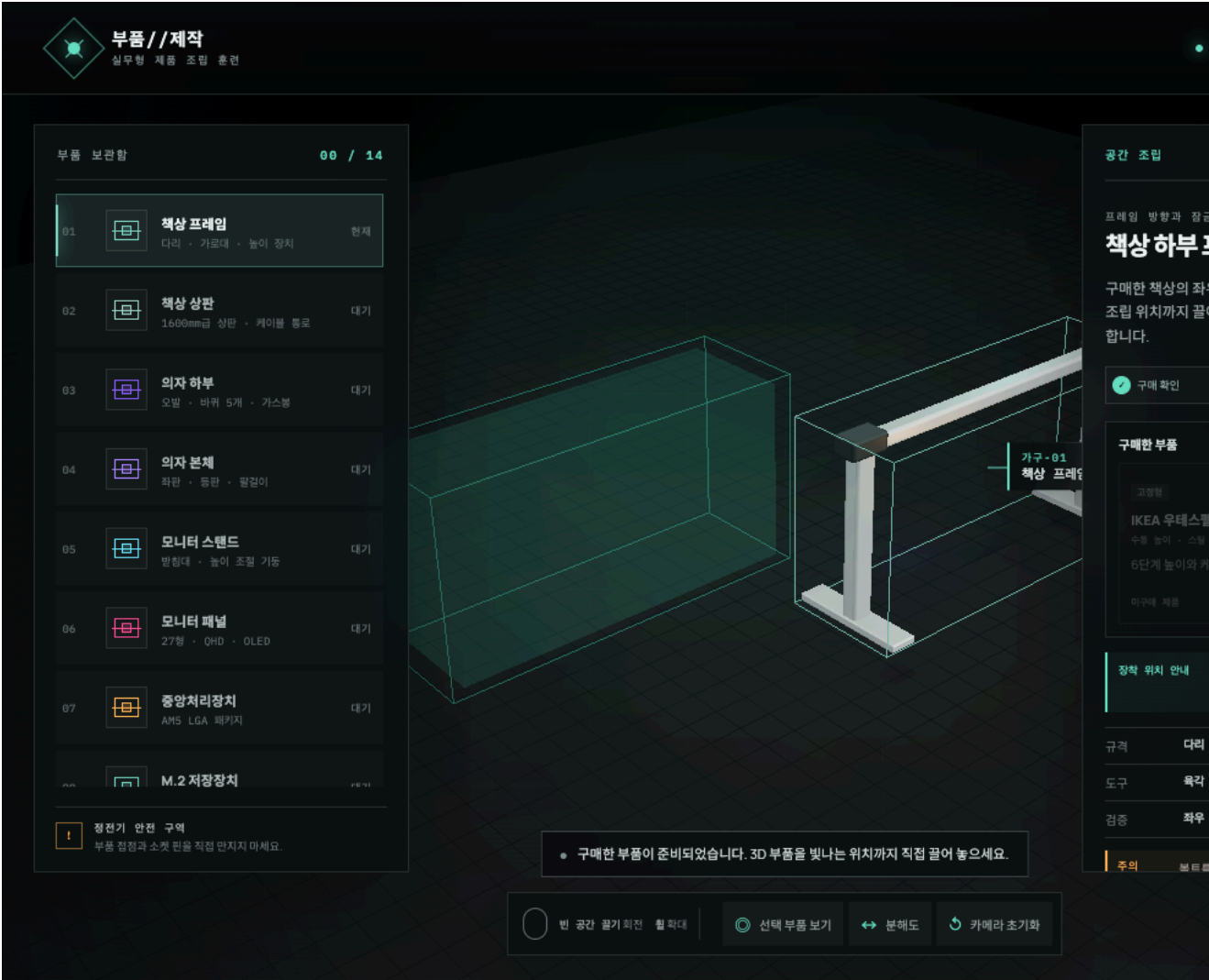
핵심 결론

부품제작 은 AI를 여섯 영역에 사용했습니다.

- 1. 문제 정의: 막연한 제품 조립 아이디어를 현실의 구매·호환·안전 문제로 좁힘
- 2. 공식 정보 조사: 제조사 사양·설명서·안전 자료를 판정 가능한 데이터로 구조화
- 3. 게임 구현: Three.js 직접 조작, 제품별 진행 규칙, 애니메이션과 피드백 제작
- 4. 3D 자산 제작: Neural4D 고밀도 메시와 Blender의 조립 가능한 노드 구조 결합
- 5. 시각 제작: 여러 생활 실습을 하나의 공간으로 설명하는 로비 이미지 생성·최적화
- 6. 검증과 개선: 자동 테스트, 실제 브라우저 QA, 성능 예산, 화면별 회귀 검사

게임 실행 중에는 외부 생성형 AI API를 호출하지 않습니다. 제품 호환과 안전 절차는 출처가 있는 JSON 데이터와 명시적인 규칙으로 판정합니다. AI의 자연어 답변을 안전 관련 정답으로 사용하지 않았습니다. 이 선택은 AI를 적게 쓴 것이 아니라 **생성에 강한 영역과 확정적이어야 하는 영역을 분리한 설계**입니다.

AI 도구	실제 사용 영역	산출물	제가 직접 내린 판단
OpenAI Codex	아이디어 구체화, 웹 조사, 코드·테스트·문서 작성, 브라우저 QA	Vite·Three.js 게임, JSON 데이터, Blender Python, 테스트, 제출 문서	주제 선택, 범위 삭제·확장, 시각 품질 판정, 실패 원인 재지시
Neural4D 2.5	강화유리 ATX 케이스의 고밀도 PBR 초안 생성	약 100만 면·4K PBR GLB 원본	실제 ATX 비례, 사용할 표면과 버릴 구조, 게임용 경량화 기준
OpenAI 이미지 생성	네 가지 생활 실습을 한 장면으로 묶는 로비 비주얼	16:9 로비 배경 원본	한국 가정 맥락, UI가 읽히는 크롭·명암, AVIF 최적화
Blender + AI 작성 스크립트	부품·재질·노드 생성, Neural4D 결합, GLB 출력	편집 가능한 <code>.blend</code> , 웹·Unreal 공용 GLB	부품 이름과 설치 계약, 좌표, 유리·팬·케이블 상태 검사



AI 활용의 결과는 문서 속 아이디어가 아니라 브라우저에서 직접 움직이고 검증되는 게임입니다.

1. AI를 어떻게 디렉팅했는가

프롬프트보다 중요한 것은 판단 기준이었습니다

처음부터 완성된 기획서를 AI에게 입력하지 않았습니다. 실제 개발 대화는 “이건 아닌 것 같다”는 피드백이 쌓이는 과정이었습니다.

처음에는 스마트폰 부품 조립을 생각했습니다. 부품이 너무 많고 실제 사용자가 조립할 가능성이 낮다는 문제를 발견해 데스크톱 PC로 좁혔습니다. PC 장면을 만든 뒤에는 “실제 부품처럼 보이지 않는다”, “공중에서 조립하는 것 같다”, “전원 케이블 위치가 어색하다”, “텍스트가 작다”는 문제를 직접 제기했습니다. AI는 그때마다 원인을 코드와 3D 좌표에서 찾아 수정했고, 저는 다시 실행 화면으로 채택 여부를 판단했습니다.

이 과정에서 제 역할은 요구사항을 많이 적는 사람이 아니라 다음 네 가지를 반복하는 디렉터였습니다.

1. 지금 결과가 사용자의 실제 문제와 맞는지 판단
2. 잘못된 느낌을 구체적인 증상으로 언어화
3. AI가 제안한 해결책을 실행 화면과 수치로 검증
4. 통과하지 못하면 이미 만든 기능도 버리고 다시 지시

전구 모듈을 형광등으로 전면 교체한 결정이 대표적입니다. 기능이 이미 동작해도 한국 생활 맥락에 맞지 않으면 완성이 아니라고 판단했습니다.

AI 활용 전체 흐름

단계 A. 문제를 질문으로 바꿉니다

“조립 게임을 만들자”를 그대로 개발하지 않았습니다. 사용자가 실제로 돈과 시간을 잃는 지점을 질문으로 바꿨습니다.

- 무엇을 보고 내 제품을 식별하는가
- 닳은 후보 중 어떤 필드를 비교해야 하는가
- 어떤 순서와 방향을 틀리는가
- 설치 뒤 무엇을 검사해야 하는가
- 어디서 멈추고 전문가에게 맡겨야 하는가

단계 B. AI가 조사하고 구조화합니다

Codex의 웹 조사 기능으로 제조사 제품 페이지, 조립 설명서, 공공기관 안내를 확인했습니다. 조사 결과를 긴 설명문으로 저장하지 않고 게임 판정에 필요한 필드로 줄였습니다.

예를 들어 형광등은 `기술 방식 + 관 형태 + 베이스 + 소비전력 + 길이 + 색온도 + 안정기 방식`, 샤홨기 필터는 `브랜드 + 제품군 + 세대 + 장착 위치 + 형태`가 일치해야 합니다. 출처 URL은 데이터 파일 안에 함께 보존했습니다.

단계 C. AI가 초안을 구현합니다

Codex는 화면 흐름, Three.js 장면, 3D 드래그, 회전 제스처, 호환성 함수, Blender 생성 스크립트와 테스트를 만들었습니다. 저는 각 산출물을 실제 브라우저에서 확인했습니다.

단계 D. 사람의 감각을 다시 데이터로 바꿉니다

“뭔가 어색하다”는 피드백을 그대로 두지 않았습니다. 예를 들어 공중 조립은 케이스와 작업대의 월드 좌표 차이로, 케이블 이상은 설치 완료용 메시지를 대기 부품에도 재사용한 구조로, 버튼 잠림은 고정 높이 그리드의 암시적 `auto` 행 문제로 좁혔습니다.

단계 E. 테스트가 결과를 고정합니다

한 번 좋아 보이는 화면으로 끝내지 않았습니다. 부품 수, 호환 제품 집합, 필수 3D 노드, 모델 용량, 모바일 자산, 지연 로딩, 패널 스크롤과 구매 버튼 가시성까지 자동 검증에 넣었습니다.

제작 파이프라인

입력	AI 작업	사람의 통과 기준	저장되는 결과	검증
생활 문제	사용자 실수·안전 경계 조사	집에서 직접 할 수 있는가	콘텐츠 범위와 상태 흐름	플레이 완주
공식 제품 자료	사양 필드 추출·비교	출처와 판정 이유가 분명한가	<code>public/data/*.json</code>	정답·유사 오답 집합 테스트
자연어 상호작용 요구	Three.js 코드 생성	직접 조작감과 복구 가능성	<code>src/*.js</code>	브라우저 드래그 QA
케이스 이미지·설명	Neural4D 고밀도 생성	표면 품질과 실제 비례	원본 GLB	Blender 시각 확인
고밀도 3D 원본	Blender 결합·압축 스크립트	노드 이름과 설치 좌표 유지	<code>.blend</code> , 최적화 GLB	필수 노드·용량 테스트
사용자 피드백	원인 분석·수정안 구현	실제 화면에서 문제가 사라졌는가	코드·CSS·자산 수정	회귀 테스트와 캡처

2. 실제 프롬프트와 판단 기록

기록 방식

아래 인용문은 실제 개발 대화에서 사용한 주요 사용자 지시를 핵심 문장 그대로 정리한 것입니다. 비밀키와 개인 정보는 제외했습니다. 중요한 것은 프롬프트의 문장력이 아니라 첫 결과 뒤에 어떤 판단을 추가했는 것입니다.

P-01. 제품 조립을 실무 문제로 만들기

“핸드폰 안에 들어가는 부품들을 레고처럼 설명서대로 만드는 거야. 그런데 스마트폰은 부품이 너무 많아. 내가 원하는 건 실무에 도움이 되는 제품 만들기 시뮬레이션이야.”

AI가 한 일: 조립 가능한 후보를 비교하고 데스크톱 PC를 첫 수직 슬라이스로 제안했습니다. 실제 ATX 부품 구성을 조사해 데이터와 단계로 만들었습니다.

제가 한 판단: 단순한 물체 제작보다 사용자가 현실에서 “컴퓨터를 맞추다”고 표현하는 구매·조립 경험이 더 적합하다고 선택했습니다.

결과물: `public/data/desktop-atx.json`, `public/data/store-catalog.json`, PC 구매·조립 흐름.

P-02. 자동 설치를 직접 조작으로 바꾸기

“마우스로 클릭해서 끌어 이동하고 사용자가 직접 해당 위치에 설치했으면 좋겠어. 어디에 설치해야 하는지 안내하고, 한 부품도 여러 종류가 있으면 좋겠어.”

AI가 한 일: 레이캐스팅 선택, 화면 좌표 기반 드래그, 목표 홀로그램, 거리·방향 판정, 실패 시 복귀, 변형 선택 구조를 구현했습니다.

제가 한 판단: 안내가 많아도 설치가 자동이면 실습이 아니라고 보았습니다. 정답 위치는 보이되 마지막 이동은 사용자가 하도록 유지했습니다.

결과물: `beginPartDrag`, `movePartDrag`, `completeDraggedInstall`, 부품별 두 개 이상의 변형.

P-03. 구매를 3D 결과와 연결하기

“모니터부터 데스크톱 PC, 의자 등을 먼저 구매하고 조립한 뒤, 완성되면 방에 실제로 적용된 모습을 보여줘. 구매한 것에 따라 텍스트뿐 아니라 부품도 바뀌어야 해.”

AI가 한 일: 12개 구매 카테고리, 28개 후보, 실시간 총액, SKU별 3D 변형과 완성 방 적용을 연결했습니다.

제가 한 판단: 구매 화면을 별도 미니게임으로 두지 않고 이후 조립의 입력값으로 사용했습니다. “내가 고른 것이 정말 들어갔다”는 확인이 보상이라고 판단했습니다.

결과물: `buildShopCategories`, `applyPurchasedAppearances`, `buildRoomScene`, 최종 게이밍 룸.

P-04. “실제 같지 않다”를 3D 파이프라인으로 해결하기

“실제 부품 같지가 않아서 아쉬운데 어떻게 하지.”

“Neural4D를 이용해서 3D 에셋의 퀄리티를 높여줘.”

AI가 한 일: 검은 강화유리 ATX 미들타워를 위한 Neural4D 생성 설명을 정리하고, 고밀도 PBR GLB를 Blender 장면에 결합하는 스크립트를 작성했습니다.

제가 한 판단: 고밀도 메시 전체를 그대로 쓰지 않았습니다. 외부 셀·유리는 Neural4D의 표면 품질을 사용하고, CPU·메인보드·GPU·케이블·팬처럼 상호작용해야 하는 부품은 안정적인 이름과 좌표를 가진 프로젝트 메시로 유지했습니다.

결과물: `assets/neural4d/atx-case-source.glb`, `assets/pc-lab-source.blend`, `tools/build_pc_scene.py`, `public/models/pc-lab*.glb`.

Neural4D 핵심 생성 지시는 다음처럼 재구성했습니다.

“검은색 강화유리 ATX 미들타워 케이스. 측면과 전면 유리, 실제 금속 패널 두께, 내부 프레임과 통풍구, PBR 금속·유리 재질, 제품 촬영 수준의 비례와 디테일.”

P-05. 생활 맥락이 틀리면 완성 기능도 버리기

“우리나라는 전구가 아니라 형광등이야. 전구 대신 형광등으로, 방은 유지하되 설치 방법과 관련 내용을 모두 바꿔줘.”

AI가 한 일: 기존 전구 실습을 형광등 데이터·UI·회전 제스처·점등·분리배출 흐름으로 교체했습니다.

제가 한 판단: 이미 만든 기능을 조금 고쳐 재사용하는 대신, 한국 주거 환경에서 실제로 혼동하는 직관 램프 규격과 안정기 문제를 새로 조사하게 했습니다.

결과물: `src/fluorescent-module.js`, `public/data/fluorescent-lab.json`, 형광등 7단계 실습.

P-06. 같은 드래그 게임을 복제하지 않기

“샤워기 필터는 본체를 살피 어떤 종류인지 파악하고 필요한 필터를 사야 될 것 같아.”

“서랍같이 좀 더 대중적인 사람들이 조립 가능한 것.”

AI가 한 일: 샤워기에는 본체 세대·필터 위치·O링·누수 검사를, 서랍장에는 판재 방향·캠락·좌우 레일·대각선 직각·전도 방지를 설계했습니다.

제가 한 판단: 콘텐츠 수를 늘리는 것보다 제품마다 다른 실패 이유가 있어야 한다고 요구했습니다. 싱크대처럼 전문 수리 범위가 큰 후보는 제외했습니다.

결과물: `src/shower-filter-module.js`, `src/drawer-module.js`, 각각의 JSON 데이터와 완료 장면.

P-07. 읽기 쉽게 만들되 3D를 가리지 않기

“전체적으로 텍스트가 작아서 잘 안 보여. 하지만 메인 기능은 조립이라 텍스트가 화면 비중을 많이 가져가면 안 될 것 같아.”

AI가 한 일: 글자 크기를 올리고, 세부 내용은 패널 내부 스크롤로 분리했으며, 모바일 서랍 실습에는 접을 수 있는 가이드를 만들었습니다.

제가 한 판단: 글자를 크게 만드는 단순 수정은 3D 작업 공간을 줄입니다. 제목과 현재 행동은 고정하고 세부 설명만 점진적으로 열리도록 했습니다.

결과물: 반응형 CSS, 스크롤 가능한 상세 패널, 모바일 접기 UI, 가독성 브라우저 QA.

P-08. 마지막 날에도 실행 경로를 다시 검증하기

“게이밍 컴퓨터 조립에서 구매 단계 다음으로 넘어가는 버튼이 하단에 잘려서 안 보여.”

AI가 한 일: 1440×900, 1366×768, 1280×720, 1024×640에서 실제 DOM 좌표를 측정했습니다. 견적 내용이 그리드의 암시적 `auto` 행을 865px까지 늘려 버튼을 화면 아래로 미는 원인을 찾았습니다.

제가 한 판단: 버튼을 위로 옮기는 임시 수정 대신, 구매 목록만 내부 스크롤되고 총액·주요 행동은 항상 남는 구조로 고쳤습니다.

결과물: `grid-template-rows: minmax(0, 1fr)`, 회귀 테스트 1개 추가, 전체 22개 테스트 통과, GitHub Pages 재배포.

3. 기술 파이프라인

런타임 구조

파일·디렉터리	역할
<code>src/main.js</code>	로비와 실습별 자연 로딩, 실패·재시도 상태
<code>src/app.js</code>	공통 Three.js 장면, PC 구매·14단계 조립, 카메라·완성 연출
<code>src/fluorescent-module.js</code>	규격·안정기 호환, 90° 교체, 점등·분리배출
<code>src/shower-filter-module.js</code>	본체·리필 호환, 360° 캡, O링·누수 검사
<code>src/drawer-module.js</code>	판재 방향, 캠락, 레일, 직각, 전도 방지, 동작 시험
<code>public/data/*.json</code>	제품 사양, 단계, 안전 중단 조건, 원문 출처
<code>tests/*.test.mjs</code>	기능·자산·성능·반응형 구조 회귀 테스트
<code>tools/cdp-*.mjs</code>	실제 Chromium에서 드래그·완료·성능을 검증하는 QA 러너
<code>.github/workflows/pages.yml</code>	테스트·빌드·성능 검사 후 GitHub Pages 배포

제품 정보가 게임 규칙이 되는 과정

AI 조사 결과를 그대로 플레이어에게 보여주지 않았습니다. 다음 네 층으로 분리했습니다.

- 출처 층:** 제조사 제품 페이지와 설명서 URL
- 데이터 층:** 이름, 규격, 세대, 가격, 장착 위치, 안전 조건
- 판정 층:** 필드 단위의 결정적 호환성 함수
- 피드백 층:** 틀린 필드를 플레이어가 이해할 수 있는 한국어로 변환

예를 들어 샤워기 필터의 브랜드만 맞고 제품군이나 세대가 다르면 오답입니다. 형광등은 G13 핀이 같아도 길이·소비전력·안정기 방식이 다르면 설치할 수 없습니다. AI가 조 사한 지식을 **설명 가능한 게임 규칙**으로 바꾼 부분입니다.

Neural4D × Blender 하이브리드 자산

Neural4D 원본은 약 100만 면, 27MB 규모의 4K PBR GLB였습니다. 표면 품질은 좋았지만 웹 조립에 그대로 쓰기에는 무거웠고, `part_gpu` 처럼 게임 코드가 찾아야 하는 부품 이름과 충돌 영역도 없었습니다.

따라서 다음 파이프라인을 만들었습니다.

- Neural4D GLB를 Blender에 가져옵니다.
- 생성 모델의 축과 실제 ATX 비례를 작업대 좌표에 맞춥니다.
- 케이스 외부 셀과 표면 디테일은 Neural4D 메시를 사용합니다.
- 조립 부품은 프로젝트 전용 계층과 결정적인 노드 이름을 유지합니다.
- 강화유리의 투명·반사 재질과 금속 표면을 별도로 설정합니다.
- 데스크톱 모델은 약 6.3MB, 모바일 모델은 약 3.2MB로 출력합니다.
- Meshopt 압축 뒤에도 필수 부품 노드가 남는지 테스트합니다.



생성형 3D의 표면 품질과 게임 코드가 요구하는 안정적인 상호작용 구조를 분리했습니다.

설치 전후 상태를 분리한 케이블

초기 케이블 부품은 설치 완료 뒤의 긴 라우팅 메시지를 대기 상태에서도 사용했습니다. 그래서 사용자가 잡기 전부터 케이블이 케이스를 관통하고 공중에 떠 보였습니다.

AI와 함께 원인을 메시 경계와 월드 좌표로 추적한 뒤 다음처럼 분리했습니다.

- 설치 전: 작업대 위에 묶인 짧은 케이블 코일
- 드래그 중: 사용자가 잡을 수 있는 간단한 상호작용 프록시
- 설치 후: 메인보드·CPU·GPU·저장장치로 연결된 전체 라우팅

이 수정은 시각적 문제를 숨긴 것이 아니라 하나의 부품이 게임 상태에 따라 다른 형상을 가져야 한다는 모델링 계약을 세운 작업입니다.

로딩과 성능 최적화

처음에는 로비 진입 시 Three.js와 모든 실습 코드를 함께 불러왔습니다. 실제 브라우저 성능 측정 뒤 다음을 적용했습니다.

- 실습을 선택할 때만 각 모듈을 동적 import
- 로비 진입 번들과 3D 엔진·후처리·실습 모듈 분리
- 데스크톱·모바일 GLB 선택
- 4K 원본을 유지하면서 웹용 2K 텍스처와 Meshopt 압축 출력
- 96KB AVIF 로비 이미지 우선 로딩
- 모델·청크·로비 이미지에 성능 예산 설정

현재 프로덕션 빌드는 10개 청크로 분리됩니다. 초기 진입 코드와 무거운 3D 장면을 분리해, 네 콘텐츠가 있다는 이유로 첫 화면을 오래 기다리지 않게 했습니다.

4. 실패, 검증, 책임

AI의 첫 답을 그대로 쓰지 않은 기록

관찰한 문제	첫 결과의 한계	수정한 구조	남긴 증거
PC가 공중에서 조립되는 느낌	모델과 작업대 좌표 기준이 달랐음	조립 루트·바닥·카메라 기준 재정렬	전후 QA 캡처
케이블 위치가 어색함	완료용 긴 메시지를 대기 상태에도 사용	loose-installed 메시 분리	케이블 디버그·회귀 테스트
부품이 실제 같지 않음	절차형 기본 도형의 표면 디테일 부족	Neural4D 셀 + Blender 상호작용 노드	원본 GLB·BLEND·최적화 GLB
케이스가 뚫려 보임	유리 패널과 반사·투명 재질 부족	측면·전면 강화유리 메시 추가	조립 화면 캡처
텍스트가 작음	정보량을 작은 글자로 해결	고정 핵심 정보 + 내부 스크롤·접기	데스크톱·모바일 가독성 QA
첫 화면이 무거움	모든 3D 모듈을 초기 로딩	모듈 지연 로딩·체크 분리	성능 예산 검사
구매 버튼이 화면 밖으로 밀림	그리드 행이 건적 고유 높이까지 확장	1fr 행 고정 + 목록만 스크롤	4개 화면 높이 측정·22번째 테스트

저는 이 표가 AI 활용의 가장 솔직한 증거라고 생각합니다. 생성 결과가 처음부터 완벽했다면 디렉팅도 필요하지 않았을 것입니다. **AI의 속도를 믿되, 결과의 진실성은 실행 화면과 테스트로 판단했습니다.**

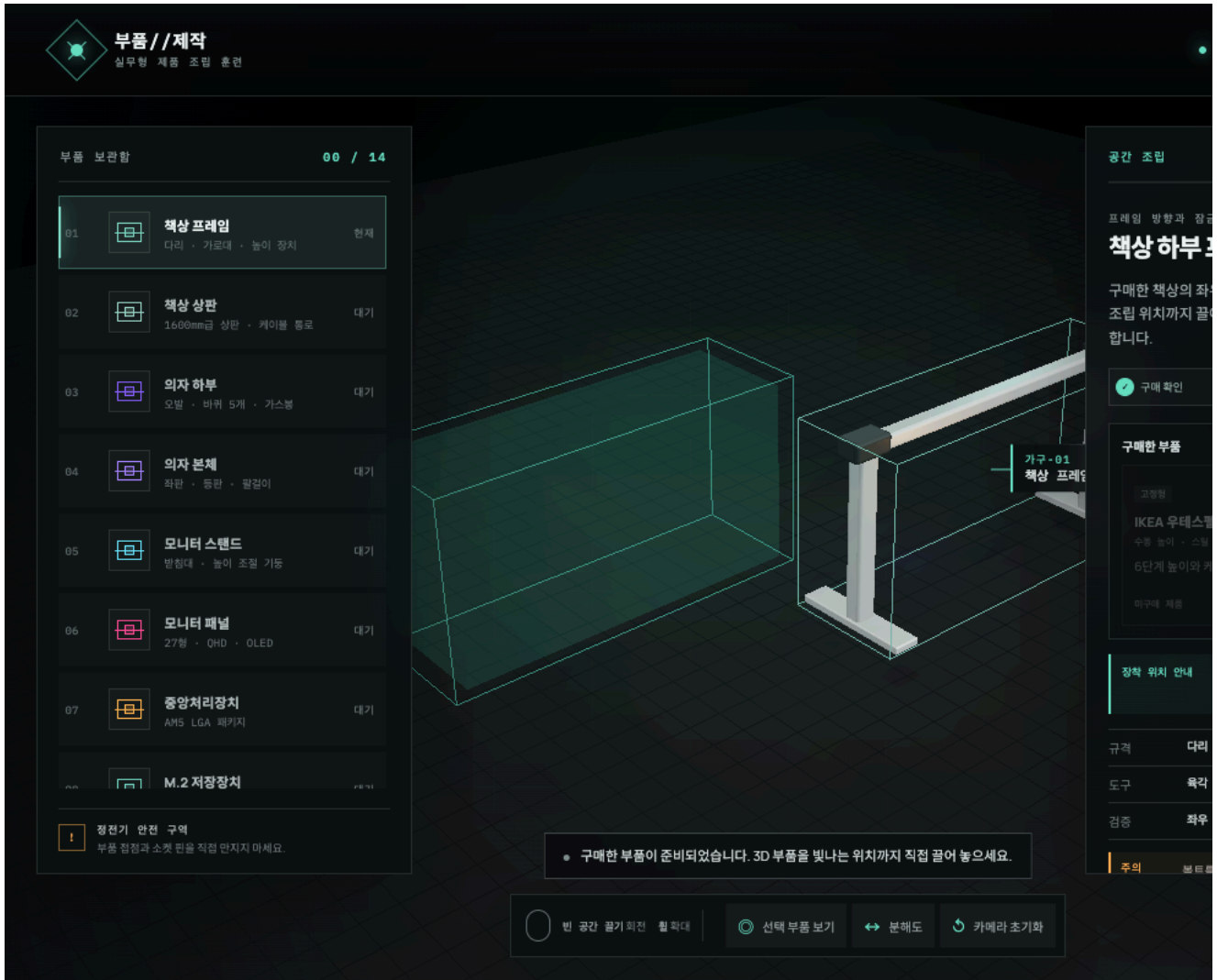
자동 검증

현재 `npm run verify` 는 프로덕션 빌드, 성능 예산, 22개 자동 테스트를 실행합니다.

주요 검사 항목은 다음과 같습니다.

- PC 14단계와 12개 구매 카테고리·28개 후보
- RTX 4070 계열을 포함한 GPU 여섯 변형
- 두 개의 정답 형광램프와 유사 오답 제품 구분
- 두 개의 정답 사워기 리필과 타세대·타제품군 구분
- 서랍장 8단계, 좌우 레일, 대각선 직각, 전도 방지
- Neural4D·Blender 원본과 Meshopt GLB 필수 노드
- 데스크톱·모바일 렌더링 자산과 성능 예산
- 모듈별 지연 로딩과 로딩 실패 후 재시도
- 모바일에서 3D 작업 공간을 남기는 접이식 가이드
- 구매 화면의 주요 행동 버튼 가시성

정적 검사만으로 확인할 수 없는 드래그, 팬 회전, 카메라 이동, 점등, 누수와 서랍 움직임은 Chrome DevTools Protocol 기반 QA 러너로 실제 Chromium에서 재현하고 화면을 저장했습니다.



사람과 AI의 역할 구분

단계	사람, 이윤수	AI
문제 선택	집에서 실제로 겪는 문제와 제외할 전문 수리 범위 결정	후보 조사와 게임 상태 분해
제품 범위	PC → 형광등 → 사위기 → 서랍장으로 확장 결정	사양·절차·안전 자료 구조화
상호작용	직접 움직여야 한다는 원칙과 조작감 채택 여부 판단	드래그·회전·스냅·카메라 코드 작성
시각 품질	공중 조립, 비현실적 부품, 유리-케이블 문제 제거	3D 생성-Blender 후처리-재질 수정
사용성	작은 글자와 가려진 버튼을 실제 화면에서 발견	레이아웃 원인 분석과 회귀 수정
최종 책임	출처·범위·배포·제출 여부 승인	문서화와 반복 가능한 검사 제공

AI는 많은 코드를 빠르게 만들었습니다. 그러나 “어떤 문제를 만들 가치가 있는가”, “이 결과가 실제로 느껴지는가”, “사용자가 어디서 멈춰야 하는가”는 사람이 결정했습니다. 이 분담이 제가 생각하는 AI 디렉터의 역할입니다.

보안과 재현성

- 저장소와 프로덕션 번들에 OpenAI-Neural4D API 키를 포함하지 않았습니다.
- 개발 대화의 비밀값은 프롬프트 기록과 제출 문서에서 제거했습니다.
- 게임 실행 중 외부 AI API를 호출하지 않습니다.
- 제품 데이터와 판정 규칙은 저장소에서 심사자가 확인할 수 있습니다.
- Git 커밋 19개가 아이디어 확장, 실패 수정, 성능 개선과 문서화를 순서대로 보존합니다.
- `npm install`, `npm run verify` 로 빌드와 테스트를 재현할 수 있습니다.
- GitHub Pages 배포는 GitHub Actions 최소 권한과 OIDC를 사용합니다.

외부 도구·자산·권리 표시

항목	사용 범위	권리·라이선스 처리
Three.js 0.179.1	WebGL 3D 렌더링·상호작용	MIT License
Vite 7.3.6	개발 서버·프로덕션 번들	MIT License
Blender	3D 편집·Python 자산 파이프라인	Blender GPL, 생성 프로젝트 출력물은 별도
OpenAI Codex·이미지 생성	코드·조사·문서·로비 이미지 제작 보조	OpenAI 이용 조건 확인, 제3자 권리 별도 검토
Neural4D 2.5	ATX 케이스 GLB 원본	생성 당시 서비스 약관과 계정 사용 범위 확인 필요

제품명과 상표의 권리는 각 권리자에게 있습니다. 구매 화면의 참고 이미지는 제품 식별을 위한 사전 과제 프로토타입에 사용했고 원문 URL을 데이터와 `submission/ASSET_SOURCES.md`에 기록했습니다. 별도 오픈 라이선스가 확인되지 않은 이미지는 공개·상용 전환 전에 권리자 허락을 받거나 직접 제작·정식 라이선스 자산으로 교체해야 합니다. 이 한계를 숨기지 않는 것이 AI로 빠르게 제작할 때 필요한 책임이라고 판단했습니다.

현재 한계와 다음 단계

현재 AI는 제작 파이프라인에서 사용되며 플레이 도중 자유 답변을 생성하지 않습니다. 그래서 심사 환경에서 네트워크·API 비용·확률적 응답에 의존하지 않고, 같은 입력에 같은 안전 판정을 냅니다. 반면 사용자의 반복 실수 유형을 장기적으로 학습하거나 자유로운 질문에 답하지는 못합니다.

웹에서 검증한 GLB와 부품 ID를 Unreal에서도 이어갈 수 있도록 프로젝트 골격과 Python 임포트 스크립트를 포함했지만, 현재 개발 환경에는 Unreal Editor가 없어 에디터 내 실행은 검증하지 못했습니다. 이를 완성 기능으로 과장하지 않고 후속 제작을 위한 연결 지점으로만 남겼습니다.

다음 버전의 목표는 **정답을 알려주는 챗봇이 아니라 실수 패턴을 읽는 조립 코치**입니다.

- 사용자가 놓은 위치의 거리·방향·재시도 순서를 로컬 이벤트로 기록
- “정답 위치” 대신 “지금 어떤 개념을 혼동하는지”를 분류
- 검증된 제품 규칙과 출처를 먼저 통과시킨 뒤 개인화된 힌트 생성
- 안전 관련 답변은 생성 모델 단독으로 확정하지 않고 중단 조건을 우선 적용
- 사용자가 실제 제품 라벨을 촬영하면 민감정보를 제거하고 후보 규격만 추출

제가 NAN 2026에서 증명하고 싶은 것

공식 안내는 “AI의 다음 단계를 설계할 디렉터”를 찾는다고 말합니다. 저는 이 사전 과제를 통해 AI에게 한 번의 멋진 프롬프트를 주는 능력보다, **실제 결과를 보고 틀린 점을 말하고 다시 만들게 하며 품질을 숫자와 테스트로 고정하는 능력**을 보여주고 싶었습니다.

스마트폰이라는 첫 아이디어를 버린 일, 전구 기능을 형광등으로 다시 만든 일, Neural4D의 고밀도 결과를 그대로 쓰지 않고 상호작용 구조와 결합한 일, 마지막 날 잘린 버튼 하나까지 재현해 고친 일은 모두 같은 태도에서 나왔습니다.

AI는 제 작업을 대신 끝내지 않았습니다. 대신 제가 더 많은 가설을 시험하고, 더 빨리 틀리고, 더 구체적으로 다시 질문하게 했습니다.

본선에 진출한다면 저는 48시간 동안 AI에게 기능 수를 경쟁시키지 않겠습니다. 먼저 플레이어가 어디서 망설이고 무엇을 오해하는지 관찰한 뒤, AI의 생성 능력·조사 능력·검증 능력을 각각 가장 맞는 자리에 배치하겠습니다. `부품제작`은 그 디렉팅 방식이 실제 게임으로 작동한다는 첫 증거입니다.

참고 링크

- [NAN 2026 공식 사이트](#)
- [Three.js](#)
- [Vite](#)
- [Blender](#)
- [OpenAI 이용약관](#)
- 전체 제품·안전·외부 자산 출처: 저장소의 `submission/ASSET_SOURCES.md`